

BASH SCRIPTING

Student Practical Lab

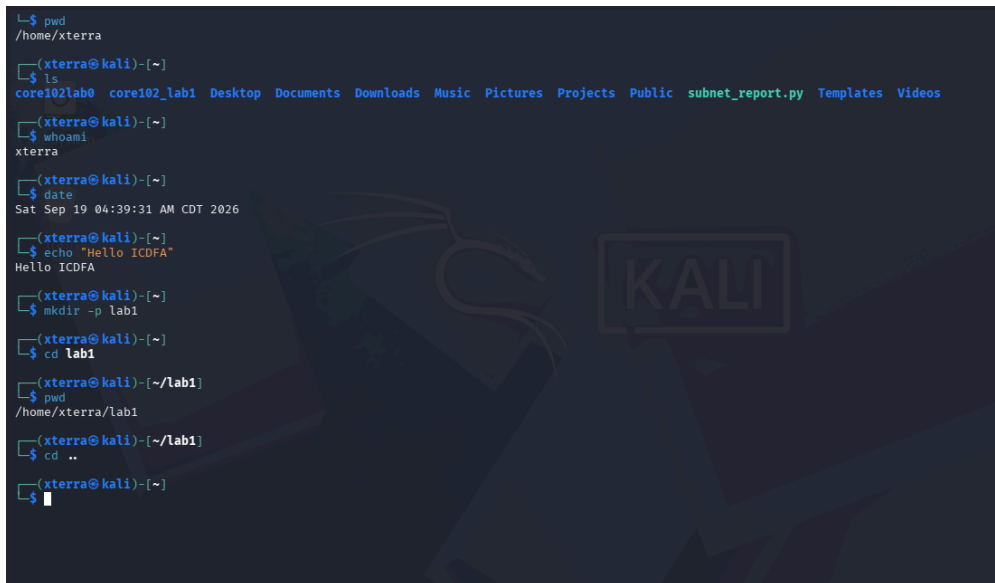
Student Name	Clement Eneji
Registration No.	C11/26/OCO/17773
Programme/Cohort	OCO/11
Instructor	Abdullahi Abdulmumin Mailafiya
Environment	Kali Linux
Date	20/09/2026
Submission	E-Campus

1. Lab 1 — Terminal Basics & Paths

Objective: Learn `pwd`, `ls`, `cd`, `clear`, `whoami`, `date` and `echo`.

Key points: `pwd` shows the current path; `ls` lists directory contents; `mkdir` creates a folder; `cd` changes location; `cd ..` returns to the parent; `clear` clears the display.

Fig. 0 — The original terminal evidence shows the practice-directory commands, `pwd`, `ls`, `whoami`, `date`, `echo`, `mkdir`, `cd` and the final working path.



```

$ pwd
/home/xterra

(xterra@kali)-[~]
$ ls
core102lab0  core102_lab1  Desktop  Documents  Downloads  Music  Pictures  Projects  Public  subnet_report.py  Templates  Videos

(xterra@kali)-[~]
$ whoami
xterra

(xterra@kali)-[~]
$ date
Sat Sep 19 04:39:31 AM CDT 2026

(xterra@kali)-[~]
$ echo "Hello ICDFA"
Hello ICDFA

(xterra@kali)-[~]
$ mkdir -p lab1

(xterra@kali)-[~]
$ cd lab1

(xterra@kali)-[~/lab1]
$ pwd
/home/xterra/lab1

(xterra@kali)-[~/lab1]
$ cd ..

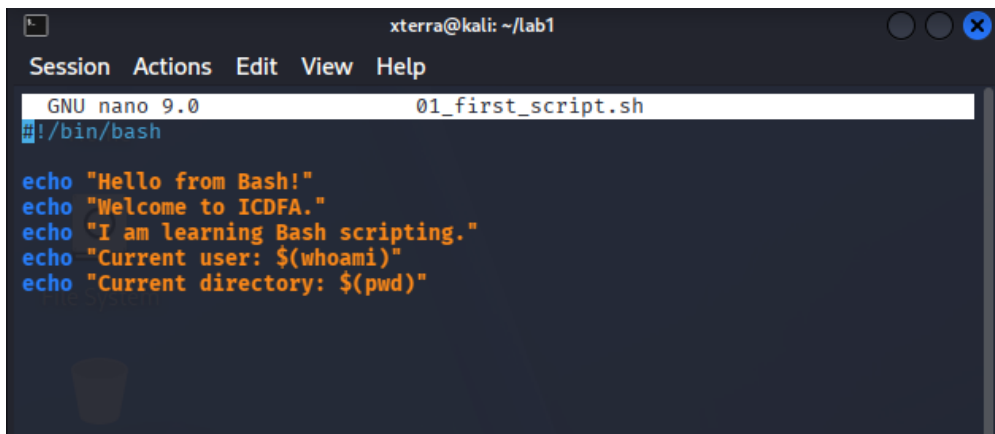
(xterra@kali)-[~]
$
```

2. Lab 2 — My First Bash Script

Objective: Create, check and run `01_first_script.sh`.

Answer: `#!/bin/bash` selects Bash. `echo` prints text, while `$(whoami)` and `$(pwd)` insert command output. `bash -n` checks syntax; `bash 01_first_script.sh` runs the script. The welcome message was changed and tested.

Fig. 1 — The original editor screenshot shows the script name and the required Bash commands, including `whoami` and `pwd` substitutions.

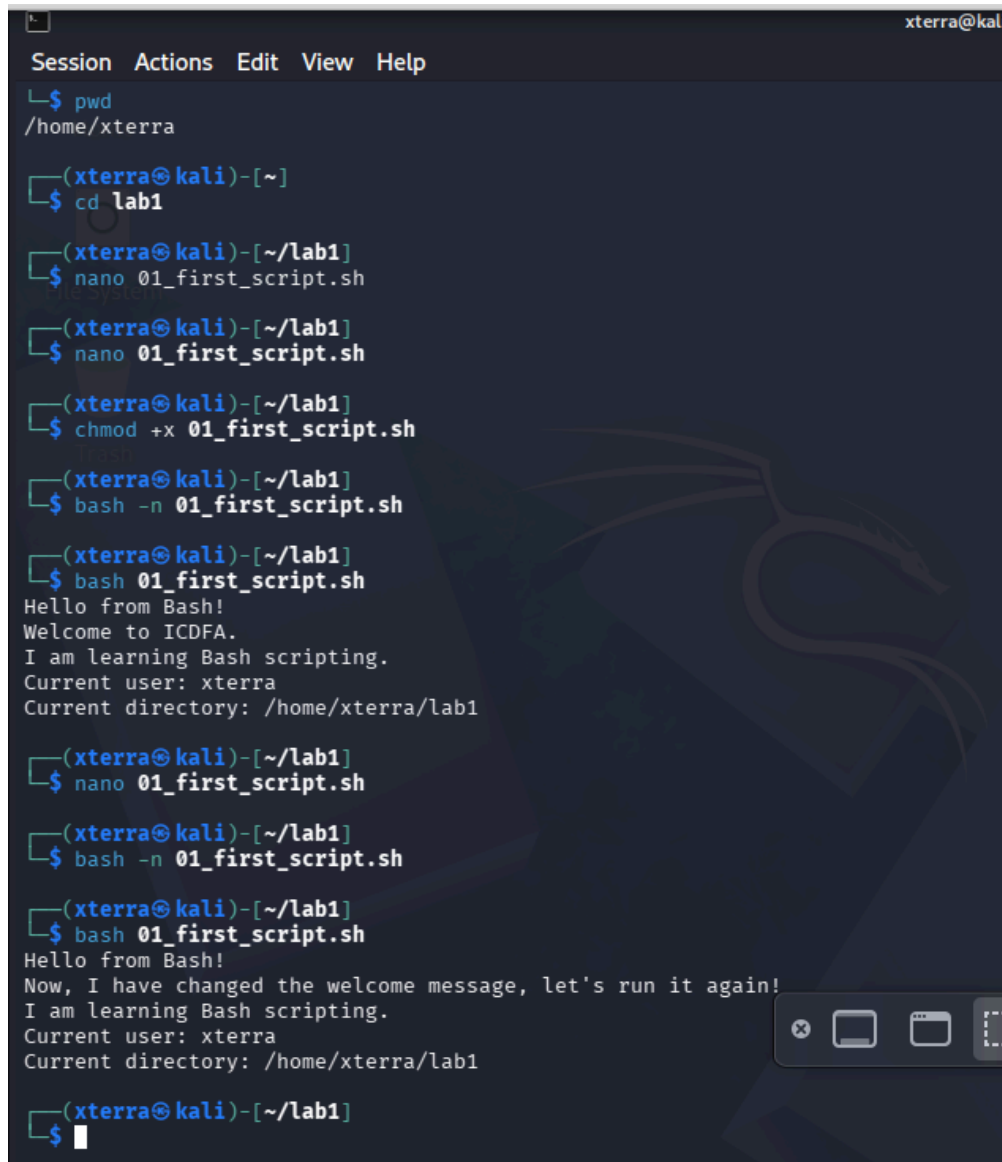


```

xterra@kali: ~/lab1
Session  Actions  Edit  View  Help
GNU nano 9.0      01_first_script.sh
#!/bin/bash

echo "Hello from Bash!"
echo "Welcome to ICDFA."
echo "I am learning Bash scripting."
echo "Current user: $(whoami)"
echo "Current directory: $(pwd)"
```

Fig. 2 — The original terminal screenshot shows syntax checking, execution, the script output, and the changed welcome message being run again.

A terminal window titled 'xterra@kali' with a menu bar (Session, Actions, Edit, View, Help). The terminal shows a series of commands and their outputs. The user starts in /home/xterra, runs 'pwd', then 'cd lab1'. They use 'nano 01_first_script.sh' to edit a file, then 'chmod +x 01_first_script.sh' to make it executable. They run 'bash -n 01_first_script.sh' for syntax checking, then 'bash 01_first_script.sh' to execute it. The output shows a welcome message. They edit the file again with 'nano 01_first_script.sh', run 'bash -n 01_first_script.sh' again, and then 'bash 01_first_script.sh' to see the updated output. The terminal has a Kali Linux dragon logo watermark and standard window controls at the bottom right.

```
Session Actions Edit View Help
└─$ pwd
/home/xterra

(xterra@kali)-[~]
└─$ cd lab1

(xterra@kali)-[~/lab1]
└─$ nano 01_first_script.sh

(xterra@kali)-[~/lab1]
└─$ nano 01_first_script.sh

(xterra@kali)-[~/lab1]
└─$ chmod +x 01_first_script.sh

(xterra@kali)-[~/lab1]
└─$ bash -n 01_first_script.sh

(xterra@kali)-[~/lab1]
└─$ bash 01_first_script.sh
Hello from Bash!
Welcome to ICDFA.
I am learning Bash scripting.
Current user: xterra
Current directory: /home/xterra/lab1

(xterra@kali)-[~/lab1]
└─$ nano 01_first_script.sh

(xterra@kali)-[~/lab1]
└─$ bash -n 01_first_script.sh

(xterra@kali)-[~/lab1]
└─$ bash 01_first_script.sh
Hello from Bash!
Now, I have changed the welcome message, let's run it again!
I am learning Bash scripting.
Current user: xterra
Current directory: /home/xterra/lab1

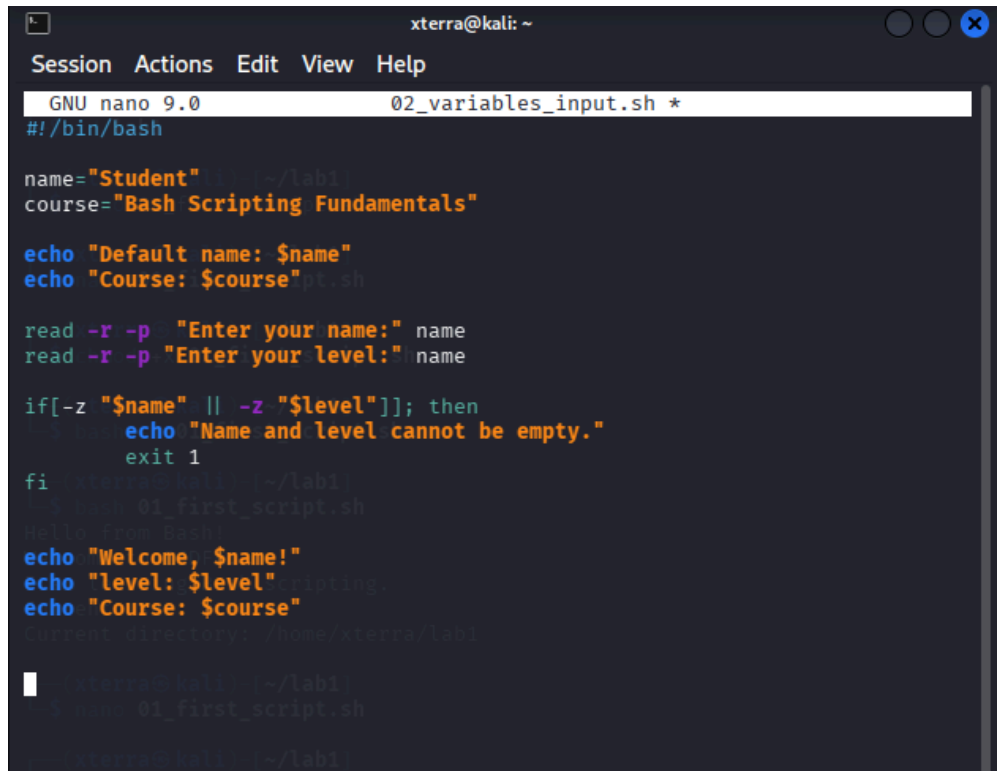
(xterra@kali)-[~/lab1]
└─$
```

3. Lab 3 — Variables & User Input

Objective: Use variables, `$variable` and `read`.

Answer: name, course and level are variables. `read -r -p` collects input. The `-z` test rejects an empty name or level and returns status 1; otherwise the entered values are displayed.

Fig. 3 — The original editor screenshot shows the variables, input prompts, empty-value test, and output statements.



```
GNU nano 9.0 02_variables_input.sh *
#!/bin/bash

name="Student"
course="Bash Scripting Fundamentals"

echo "Default name: $name"
echo "Course: $course"

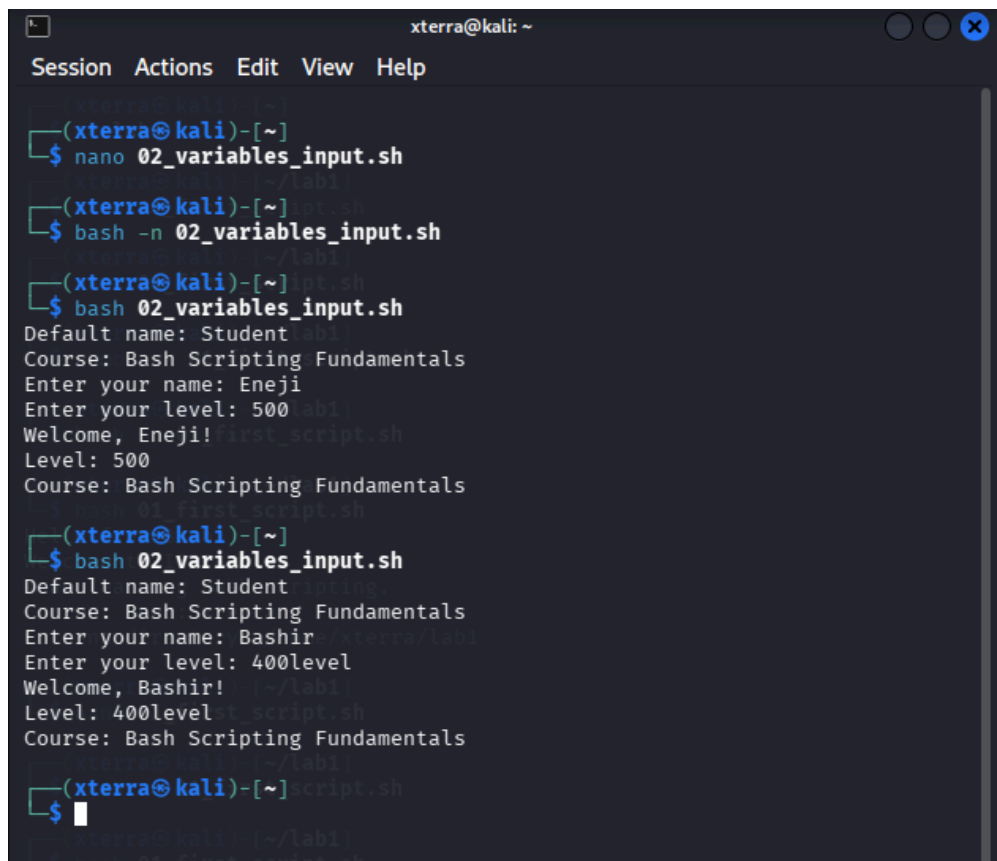
read -r -p "Enter your name:" name
read -r -p "Enter your level:" level

if [-z "$name" || -z "$level"]; then
    echo "Name and level cannot be empty."
    exit 1
fi

echo "Welcome, $name!"
echo "level: $level"
echo "Course: $course"

current_directory=$(pwd)
```

Fig. 4 — The original terminal screenshot shows syntax checking, two runs, and different entered values.



```
(xterra@kali)-[~]
$ nano 02_variables_input.sh
(xterra@kali)-[~]
$ bash -n 02_variables_input.sh
(xterra@kali)-[~]
$ bash 02_variables_input.sh
Default name: Student
Course: Bash Scripting Fundamentals
Enter your name: Eneji
Enter your level: 500
Welcome, Eneji!
Level: 500
Course: Bash Scripting Fundamentals
(xterra@kali)-[~]
$ bash 02_variables_input.sh
Default name: Student
Course: Bash Scripting Fundamentals
Enter your name: Bashir
Enter your level: 400
Welcome, Bashir!
Level: 400
Course: Bash Scripting Fundamentals
(xterra@kali)-[~]
$
```

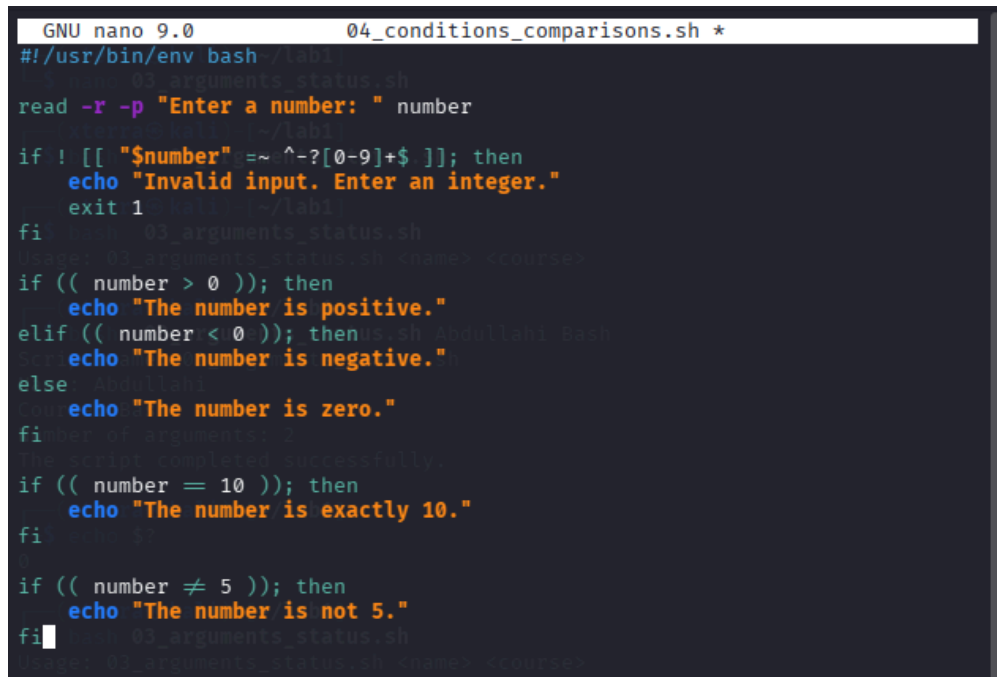
4. Lab 4 — Arguments & Exit Status

Objective: Use \$1, \$2, \$# and exit status.

Answer: \$0 is the script name, \$1 and \$2 are the first two arguments, and \$# counts all arguments. Fewer than two arguments returns status 1; a successful run returns 0. echo \$? displays the last status.

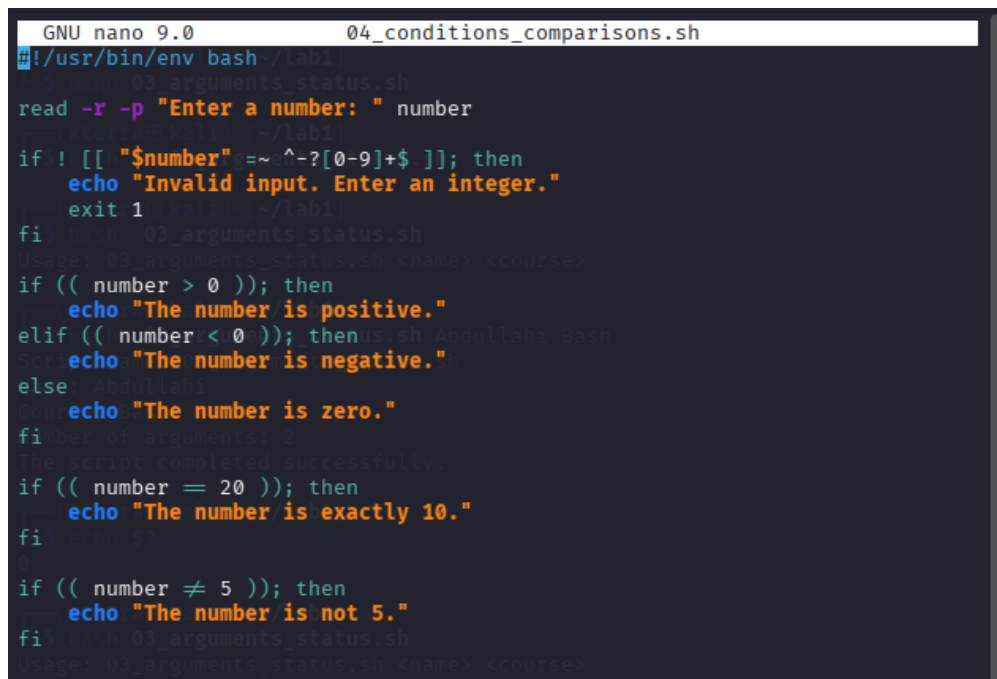
Answers: 10 is positive and exactly 10; -5 is negative; 0 is zero; and 5 is positive but is neither 10 nor “not 5”. The comparison target can be changed. The regular expression validates the integer before the if/elif/else tests run.

Fig. 7 — The original editor screenshot shows integer validation, the positive/negative/zero branches, and the comparison tests.



```
GNU nano 9.0                                04_conditions_comparisons.sh *
#!/usr/bin/env bash /lab1
# nano 03_arguments_status.sh
read -r -p "Enter a number: " number
# nano 03_arguments_status.sh
if ! [[ "$number" =~ ^-?[0-9]+$ ]]; then
    echo "Invalid input. Enter an integer."
    exit 1
fi
# nano 03_arguments_status.sh
# nano 03_arguments_status.sh
if (( number > 0 )); then
    echo "The number is positive."
elif (( number < 0 )); then
    echo "The number is negative."
else
    echo "The number is zero."
fi
# nano 03_arguments_status.sh
# nano 03_arguments_status.sh
if (( number == 10 )); then
    echo "The number is exactly 10."
fi
# nano 03_arguments_status.sh
if (( number != 5 )); then
    echo "The number is not 5."
fi
# nano 03_arguments_status.sh
# nano 03_arguments_status.sh
```

Fig. 8 — The original terminal evidence shows the script being run with the requested numeric cases.



```
GNU nano 9.0                                04_conditions_comparisons.sh
#!/usr/bin/env bash /lab1
# nano 03_arguments_status.sh
read -r -p "Enter a number: " number
# nano 03_arguments_status.sh
if ! [[ "$number" =~ ^-?[0-9]+$ ]]; then
    echo "Invalid input. Enter an integer."
    exit 1
fi
# nano 03_arguments_status.sh
# nano 03_arguments_status.sh
if (( number > 0 )); then
    echo "The number is positive."
elif (( number < 0 )); then
    echo "The number is negative."
else
    echo "The number is zero."
fi
# nano 03_arguments_status.sh
# nano 03_arguments_status.sh
if (( number == 20 )); then
    echo "The number is exactly 10."
fi
# nano 03_arguments_status.sh
if (( number != 5 )); then
    echo "The number is not 5."
fi
# nano 03_arguments_status.sh
# nano 03_arguments_status.sh
```

Fig. 9 — The original terminal evidence highlights the changed comparison/result behavior.

```
(xterra@kali)-[~] ~/lab1
$ nano 04_conditions_comparisons.sh

(xterra@kali)-[~] ~/lab1
$ bash 04_conditions_comparisons.sh
Enter a number: 20
The number is positive.
The number is exactly 10.
The number is not 5.

(xterra@kali)-[~]
$ bash 04_conditions_comparisons.sh
Enter a number: 10
The number is positive.
The number is not 5.

(xterra@kali)-[~]
$ bash 04_conditions_comparisons.sh
Enter a number: 4
The number is positive.
The number is not 5.

(xterra@kali)-[~]
$ bash 04_conditions_comparisons.sh
Enter a number: 5
The number is positive.

(xterra@kali)-[~]
$ bash 04_conditions_comparisons.sh
Enter a number: -4
The number is negative.
The number is not 5.

(xterra@kali)-[~]
$ bash 04_conditions_comparisons.sh
Enter a number: 0
The number is zero.
The number is not 5.

(xterra@kali)-[~]
$
```

6. Lab 6 — Logical AND, OR & NOT

Objective: Combine conditions with `&&`, `||` and `!`.

Answers: AND is `age >= 13 && student == yes`; OR is `age < 13 || student == no`; NOT is `! student == yes`. Non-numeric ages are rejected first.

Fig. 10 — The original screenshot shows the logical-operator script and its condition structure.

```
(xterra@kali)-[~] ~/lab1
$ nano 05_logical_operators.sh.sh

(xterra@kali)-[~] ~/lab1
$ bash -n 05_logical_operators.sh

(xterra@kali)-[~]
$ bash 05_logical_operators.sh
Enter your age: 55
Are you a student? (yes/no): yes
AND: both conditions are true.

(xterra@kali)-[~]
$ bash 05_logical_operators.sh
Enter your age: 20
Are you a student? (yes/no): no
OR: at least one condition is true.
NOT: the student answer is not yes.

(xterra@kali)-[~]
$ bash 05_logical_operators.sh
Enter your age: 10
Are you a student? (yes/no): yes
OR: at least one condition is true.

(xterra@kali)-[~]
$ bash 05_logical_operators.sh
Enter your age: 10
Are you a student? (yes/no): no
OR: at least one condition is true.
NOT: the student answer is not yes.

(xterra@kali)-[~]
$ bash 05_logical_operators.sh
Enter your age: abc
Are you a student? (yes/no): yes
Age must be a number.

(xterra@kali)-[~]
$
```

7. Lab 7 — case & Menus

Objective: Build a simple menu with case.

Answers: Options 1, 2, 3 and 0 run whoami, date, pwd and exit. Invalid input reaches *. A hostname option can be added as 4) hostname ;;.

Fig. 11 — The original editor screenshot shows the menu text, input variable and case patterns.

```
GNU nano 9.0                                06_case_menu.sh
#!/usr/bin/env bash
(xtterra@kali)~[~/lab1]
echo "==== Bash Practice Menu =====".sh
echo "1) Show user"
echo "2) Show date"
echo "3) Show current directory"1)
echo "0) Exit"03_arguments_status.sh

read -r -p "Choose an option: " choice
(xtterra@kali)~[~/lab1]
case "$choice" in
1)03_arguments_status.sh
Usage: whoami_arguments_status.sh <name> <course>
;;
2)
date
;;
3)03_arguments_status.sh Abdullahi Bash
Script name: 03_arguments_status.sh
Name: Abdullahi
Course
0)echo "Goodbye."
Number of arguments: 2
*)echo "Invalid choice."successfully.
;;
esac
(xtterra@kali)~[~/lab1]
$ echo $?
0
```

Fig. 12 — The original terminal screenshot shows menu selection and command output.

```
(xterra@kali)-[~/core102_lab1]
$ nano 06_case_menu.sh
(xterra@kali)-[~/core102_lab1]
$ bash -n 06_case_menu.sh
(xterra@kali)-[~/core102_lab1]
$ bash 06_case_menu.sh
===== Bash Practice Menu =====
1) Show user
2) Show date
3) Show current directory
0) Exit
Choose an option: 1
xterra

(xterra@kali)-[~/core102_lab1]
$ bash 06_case_menu.sh
===== Bash Practice Menu =====
1) Show user
2) Show date
3) Show current directory
0) Exit
Choose an option: 2
Sun Sep 20 08:35:09 AM CDT 2026 successfully.

(xterra@kali)-[~/core102_lab1]
$ bash 06_case_menu.sh
===== Bash Practice Menu =====
1) Show user
2) Show date
3) Show current directory
0) Exit
Choose an option: 3
/home/xterra/core102_lab1

(xterra@kali)-[~/core102_lab1]
$ bash 06_case_menu.sh
===== Bash Practice Menu =====
1) Show user
2) Show date
3) Show current directory
0) Exit
Choose an option: 0
Goodbye.

(xterra@kali)-[~/core102_lab1]
$ bash 06_case_menu.sh
===== Bash Practice Menu =====
1) Show user
2) Show date
3) Show current directory
0) Exit
Choose an option: 9
Invalid choice.

(xterra@kali)-[~/core102_lab1]
$
```

Fig. 13 — The original terminal evidence shows the remaining menu tests and invalid-choice handling.

```
GNU nano 9.0
#!/usr/bin/env bash

echo "==== Bash Practice Menu ====="
echo "1) Show user"
echo "2) Show date"
echo "3) Show current directory"
echo "0) Exit"

read -r -p "Choose an option: " choice

case "$choice" in
    1) whoami
        ;;
    2) date
        ;;
    3) pwd
        ;;
    0) echo "Goodbye."
        ;;
    *) echo "Invalid choice."
        ;;
esac

#!/usr/bin/env bash
# Number of arguments: 2
# Usage: 03_arguments_status.sh <name> <course>

echo "==== Bash Practice Menu ====="
echo "1) Show user"
echo "2) Show date"
echo "3) Show current directory"
echo "4) Show hostname"
echo "0) Exit"

read -r -p "Choose an option: " choice

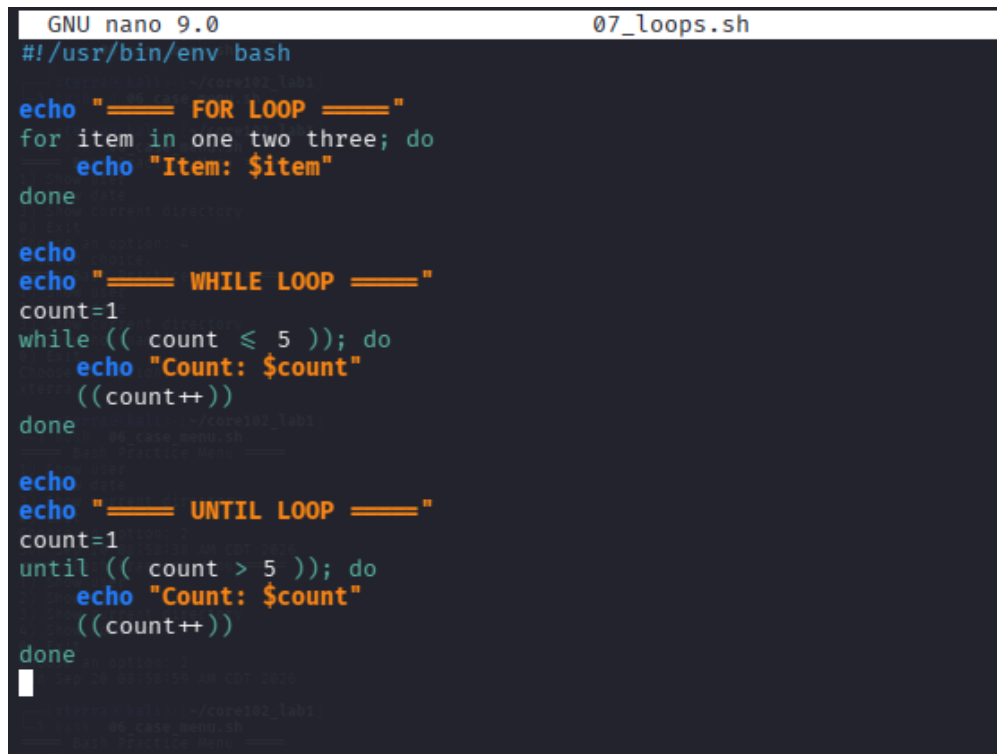
case "$choice" in
    1) whoami
        ;;
    2) date
        ;;
    3) pwd
        ;;
    4) hostname
        ;;
    0) echo "Goodbye."
        ;;
    *) echo "Invalid choice."
        ;;
esac
```

8. Lab 8 — Loops: for, while & until

Objective: Repeat tasks with three common Bash loop types.

Answers: `for` iterates through a list; `while` runs while its condition is true; `until` runs until its condition becomes true. Changing 3 to 5 gives five repetitions, and adding an item adds another `for` iteration.

Fig. 14 — The original editor screenshot shows the three loop structures and their stopping conditions.



```
GNU nano 9.0                                07_loops.sh
#!/usr/bin/env bash

echo "=== FOR LOOP ==="
for item in one two three; do
    echo "Item: $item"
done

echo
echo "=== WHILE LOOP ==="
count=1
while (( count ≤ 5 )); do
    echo "Count: $count"
    ((count++))
done

echo
echo "=== UNTIL LOOP ==="
count=1
until (( count > 5 )); do
    echo "Count: $count"
    ((count++))
done
```

Fig. 15 — The original terminal evidence shows the script check and loop output.

```
(xterra@kali)-[~/core102_lab1]
$ nano 06_case_menu.sh
(xterra@kali)-[~/core102_lab1]_status.sh
$ bash -n 06_case_menu.sh
(xterra@kali)-[~/core102_lab1]_status.sh
$ bash 06_case_menu.sh
===== Bash Practice Menu =====
1) Show user
2) Show date
3) Show current directory
0) Exit
Choose an option: 4
Invalid choice.
===== Bash Practice Menu =====
1) Show user
2) Show date
3) Show current directory
4) Show hostname
0) Exit
Choose an option: 1
(xterra@kali)-[~/core102_lab1]_status.sh Abdullahi Bash
(xterra@kali)-[~/core102_lab1]_status.sh
$ bash 06_case_menu.sh
===== Bash Practice Menu =====
1) Show user
2) Show date
3) Show current directory
0) Exit
Choose an option: 2
Sun Sep 20 08:58:38 AM CDT 2026
===== Bash Practice Menu =====
1) Show user
2) Show date
3) Show current directory
4) Show hostname
0) Exit
Choose an option: 2
Sun Sep 20 08:58:59 AM CDT 2026
(xterra@kali)-[~/core102_lab1]_status.sh
$ bash 06_case_menu.sh
===== Bash Practice Menu =====
1) Show user
2) Show date
3) Show current directory
0) Exit
Choose an option: 3
/home/xterra/core102_lab1
===== Bash Practice Menu =====
1) Show user
2) Show date
3) Show current directory
4) Show hostname
0) Exit
Choose an option: 3
/home/xterra/core102_lab1
(xterra@kali)-[~/core102_lab1]
$
```

Fig. 16 — The original screenshot shows the loop values and repeated execution.

```
Session Actions Edit View Help
GNU nano 9.0                                07_loops.sh *
#!/usr/bin/env bash

echo "==== FOR LOOP ===="
for item in one two three four five; do
    echo "Item: $item"
done

echo
echo "==== WHILE LOOP ===="
count=1
while (( count ≤ 5 )); do
    echo "Count: $count"
    ((count++))
done

echo
echo "==== UNTIL LOOP ===="
count=1
until (( count > 5 )); do
    echo "Count: $count"
    ((count++))
done
```

Fig. 17 — The original output demonstrates the for, while and until sections.

```
(xterra@kali)-[~]
$ bash 07_loops.sh
==== FOR LOOP ====
Item: one
Item: two
Item: three
Item: four
Item: five

==== WHILE LOOP ====
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5

==== UNTIL LOOP ====
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5

(xterra@kali)-[~]
$
```

Fig. 18 — The original editor evidence shows the changed loop limit and item list.

```
GNU nano 9.0                                07_loops.sh
#!/usr/bin/env bash

echo "=== FOR LOOP ==="
for item in one two three four five six; do
    echo "Item: $item"
done

echo
echo "=== WHILE LOOP ==="
count=1
while (( count ≤ 6 )); do
    echo "Count: $count"
    ((count++))
done

echo
echo "=== UNTIL LOOP ==="
count=1
until (( count > 6 )); do
    echo "Count: $count"
    ((count++))
done
```

Fig. 19 — The original terminal output confirms the changed loop behavior.

```
(xterra@kali)-[~]
$ bash 07_loops.sh
=== FOR LOOP ===
Item: one
Item: two
Item: three
Item: four
Item: five
Item: six

=== WHILE LOOP ===
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
Count: 6

=== UNTIL LOOP ===
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
Count: 6

(xterra@kali)-[~]
$
```

9. Lab 9 — Functions

Objective: Create reusable blocks of Bash code.

Answers: print_heading prints a formatted heading using \$1; greet prints a supplied name. A date function can be show_date() { date; }. Functions reduce repetition.

Fig. 20 — The original function screenshot shows `print_heading`, `greet`, input validation and system-information output.

```
GNU nano 9.0                                08_functions.sh *
#!/usr/bin/env bash

print_heading() {
    echo
    echo "=== $1 ==="
}

greet() {
    echo "Hello, $1!"
}

show_date() {
    date
}

print_heading "Functions"

read -r -p "Enter your name: " name

if [[ -z "$name" ]]; then
    echo "Name cannot be empty."
    exit 1
fi

greet "$name"

print_heading "System Information"
echo "User: $(whoami)"
echo "Host: $(hostname)"

print_heading "Date"
show_date
```

10. Lab 10 — Redirection, Pipes & Text Tools

Objective: Use `>`, `<`, `|`, `wc`, `sort`, `uniq` and `grep`.

Answers: `>` writes output to a file; `<` supplies file input; and `|` passes one command's output to another. `sort | uniq` removes duplicates, while `grep -n` finds matching lines. Another topic can be appended to the file.

Fig. 21 — The original screenshot shows the pipeline script, redirection into `topics.txt`, sorting, `uniq` and `grep`.

```

xterra@kali: ~
Session Actions Edit View Help

(xterra@kali)-[~]
$ nano 09_files_pipes_redirect.sh
(xterra@kali)-[~]
$ bash -n 09_files_pipes_redirect.sh
(xterra@kali)-[~]
$ bash 09_files_pipes_redirect.sh
Created: bash_pipeline_lab/topics.txt

===== FILE CONTENTS =====
Bash
Linux
Bash
Security
Linux

===== NUMBER OF LINES =====
5

===== SORTED UNIQUE TOPICS =====
Bash
Linux
Security

===== LINES CONTAINING LINUX =====
2:Linux
5:Linux

(xterra@kali)-[~]
$ cat bash_pipeline_lab/topics.txt
Bash
Linux
Bash
Security
Linux

(xterra@kali)-[~]
$ >
zsh: parse error near `\'n\'

(xterra@kali)-[~]
$ echo "Hello" > test.txt

(xterra@kali)-[~]
$ echo "Networking" >> bash_pipeline_lab/topics.txt

(xterra@kali)-[~]
$ cat bash_pipeline_lab/topics.txt
Bash
Linux
Bash
Security
Linux
Networking

```

11. Lab 11 — find & Command Substitution

Objective: Find files and store command output in a variable.

Answers: `$(...)` inserts command output into a command or assignment. `find ... | wc -l` counts files, while `-name "*.txt"` selects text files. Adding a `.txt` file increases the count by one.

Fig. 22 — The original editor screenshot shows `find`, the pipe to `wc -l`, command substitution and the `*.txt` filter.

```

GNU nano 9.0      10_find_command_substitution.sh
#!/usr/bin/env bash

mkdir -p practice_files

touch practice_files/one.txt
touch practice_files/two.txt
touch practice_files/notes.md

total="$(find practice_files -type f | wc -l)"

echo "Files found: $total"

===== NUMBER OF LINES =====
echo
echo "===== TEXT FILES ====="
find practice_files -type f -name "*.txt" -print

```

Fig. 23 — The original terminal evidence shows the file count and the text-file results.

```
xterra@kali: ~
Session Actions Edit View Help
(xterra@kali)-[~]
$ nano 10_find_command_substitution.sh
(xterra@kali)-[~]
$ bash -n 10_find_command_substitution.sh
(xterra@kali)-[~]
$ bash 10_find_command_substitution.sh
Files found: 3

=== TEXT FILES ===
practice_files/two.txt
practice_files/one.txt

(xterra@kali)-[~]
$ total="$(find practice_files -type f | wc -l)"
(xterra@kali)-[~]
$ touch practice_files/three.txt
(xterra@kali)-[~]
$ bash 10_find_command_substitution.sh
Files found: 4

=== TEXT FILES ===
practice_files/two.txt
practice_files/one.txt
practice_files/three.txt

(xterra@kali)-[~]
$
```

12. Lab 12 — Simple Bash Calculator

Objective: Combine input, validation, case and arithmetic.

Answers: The regular expression accepts integers. The menu performs four arithmetic operations, rejects invalid input, and prevents division by zero. A new operation can be added as another case branch.

Fig. 24 — The original calculator editor screenshot shows integer validation, the menu and arithmetic branches.

```
GNU nano 9.0 11_calculator.sh *
#!/usr/bin/env bash

read -r -p "First integer: " a
read -r -p "Second integer: " b

if ! [[ "$a" =~ ^-?[0-9]+$ && "$b" =~ ^-?[0-9]+$ ]]; then
    echo "Please enter integers only."
    exit 1
fi

echo "1) Add"
echo "2) Subtract"
echo "3) Multiply"
echo "4) Divide"

read -r -p "Choose operation: " op

case "$op" in
    1)
        echo "Result: $((a + b))"
        ;;
    2)
        echo "Result: $((a - b))"
        ;;
    3)
        echo "Result: $((a * b))"
        ;;
    4)
        if (( b == 0 )); then
            echo "Cannot divide by zero."
            exit 1
        fi
        echo "Result: $((a / b))"
        ;;
    *)
        echo "Invalid operation."
        ;;
esac
```

Help **Write Out** **Where Is** **Cut** **Execute** **Location**
Exit **Read File** **Replace** **Paste** **Justify** **Go To Line**

Fig. 25 — The original terminal evidence shows calculator execution and results.

```
Session Actions Edit View Help

(xterra@kali)-[~]
$ nano 11_calculator.sh direct.sh

(xterra@kali)-[~]
$ bash -n 11_calculator.sh direct.sh

(xterra@kali)-[~]
$ bash 11_calculator.sh direct.sh
First integer: 10 /home/xtterra/test
Second integer: 5
1) Add
2) Subtract
3) Multiply
4) Divide
Choose operation: 1
Result: 15

(xterra@kali)-[~]
$ bash 11_calculator.sh
First integer: 10
Second integer: 5 /home/xtterra/test
1) Add
2) Subtract
3) Multiply
4) Divide
Choose operation: 2
Result: 5

(xterra@kali)-[~]
$ bash 11_calculator.sh
First integer: 10 /home/lab/topics.txt
Second integer: 5
1) Add
2) Subtract
3) Multiply
4) Divide
Choose operation: 3
Result: 50

(xterra@kali)-[~]
$ bash 11_calculator.sh
First integer: 10
Second integer: 5 /test.txt
1) Add
2) Subtract
3) Multiply
4) Divide
Choose operation: 4
Result: 2 /bash_pipeline_lab/topics.txt

(xterra@kali)-[~]
$ bash 11_calculator.sh
First integer: 10
Second integer: 0
1) Add
2) Subtract
3) Multiply
4) Divide
Choose operation: 4
Cannot divide by zero.
```

Fig. 26 — The original screenshot shows the division and error-handling tests.

```
Session Actions Edit View Help

(xterra@kali)-[~]
$ bash 11_calculator.sh direct.sh
First integer: abc
Second integer: 5
Please enter integers only. /direct.sh

(xterra@kali)-[~]
$ bash 11_calculator.sh direct.sh
First integer: 10 /home/lab/topics.txt
Second integer: 5
1) Add
2) Subtract
3) Multiply
4) Divide
Choose operation: 8
Invalid operation.

(xterra@kali)-[~]
$
```

13. Lab 13 — File & Directory Manager

Objective: Practise `mkdir`, `touch`, `cp`, `mv`, `rm`, `cat` and `less`.

Answers: Run the program inside `~/icdfa_bash_lab`. `mkdir` and `touch` create items; `cp` copies; `mv` moves or renames; `cat` displays; and `less` views longer files. The script checks paths and confirms file removal.

Fig. 27 — The original editor screenshot shows the safe file-manager menu and the mkdir/touch branches.

```

Session  Actions  Edit  View  Help
GNU nano 9.8                                     12_files_
#!/usr/bin/env bash
(xterra@kali)~$ nano 09_files_pipes_redirect.sh
# Run this program only inside your dedicated practice directory.
(xterra@kali)~$ bash -n 09_files_pipes_redirect.sh
while true; do
  (xterra@kali)~$
  echo "===== 09_files_pipes_redirect.sh"
  Created: 2023-09-05 14:55:10 e_lab/topics.txt
  echo "===== FILE & DIRECTORY MANAGER ====="
  echo "===== FILE CONTENTS ====="
  Bash echo "1. mkdir - Create directory"
  Linux echo "2. touch - Create empty file"
  Bash echo "3. cp - Copy file"
  Security echo "4. mv - Move or rename"
  Linux echo "5. rm - Remove file"
  echo "===== NUMBER OF LINES ====="
  Bash echo "6. cat - Display file contents"
  Linux echo "7. less - View longer files"
  echo "===== FED UNIQUE TOPICS ====="
  Bash echo "===== "
  Linux
  Security read -r -p "What would you like to do? " choice
  echo "===== LINES CONTAINING LINUX ====="
  Bash case "$choice" in
  Linux 1)
  Bash read -r -p "Enter directory name: " dir
  Bash $ cat bash_pipeline_lab/topics.txt
  Bash if [[ -z "$dir" ]]; then
  Linux echo "FAILED: Directory name cannot be empty."
  Bash
  Security elif mkdir - "$dir" 2>/dev/null; then
  Linux echo "SUCCESS: Directory '$dir' created."
  Bash
  Bash else
  (xterra@kali)~$ echo "FAILED: Directory could not be created."
  $ >
  zsh: pfile error near '\n'
  ==
  (xterra@kali)~$
  $ echo "Hello" > test.txt
  2)
  Bash read -r -p "Enter file name: " file
  Bash $ echo "Networking" >> bash_pipeline_lab/topics.txt
  Bash
  Bash if [[ -z "$file" ]]; then
  (xterra@kali)~$ echo "FAILED: File name cannot be empty."
  $ cat bash_pipeline_lab/topics.txt
  Bash elif touch - "$file" 2>/dev/null; then
  Linux echo "SUCCESS: File '$file' created/updated."
  Bash
  Bash else
  Security
  Linux echo "FAILED: File could not be created."
  Networking
  fi
  ==

```

Fig. 28 — The original screenshot shows the copy, move and remove branches with existence checks.

```
(xterra@kali)-[~/icdfa_bash_lab]
$ nano 12_file_directory_manager.sh

(xterra@kali)-[~/icdfa_bash_lab]
$ bash -n 12_file_directory_manager.sh

(xterra@kali)-[~/icdfa_bash_lab]
$ bash 12_file_directory_manager.sh

===== FILE CONTENTS =====

Linux FILE & DIRECTORY MANAGER
=====

1. mkdir - Create directory
2. touch - Create empty file
3. cp - Copy file
4. mv - Move or rename
5. rm - Remove file
6. cat - Display file contents
7. less - View longer files
0. Exit

What would you like to do? 1
Enter directory name: lab13_test
SUCCESS: Directory 'lab13_test' created.

(xterra@kali)-[~/]
=====

Bash FILE & DIRECTORY MANAGER
=====

1. mkdir - Create directory
2. touch - Create empty file
3. cp - Copy file
4. mv - Move or rename
5. rm - Remove file
6. cat - Display file contents
7. less - View longer files
0. Exit

What would you like to do? 2
Enter file name: test.txt_pipeline_lab/topics.txt
SUCCESS: File 'test.txt' created/updated.

$ cat bash_pipeline_lab/topics.txt

Linux FILE & DIRECTORY MANAGER
=====

1. mkdir - Create directory
2. touch - Create empty file
3. cp - Copy file
4. mv - Move or rename
5. rm - Remove file
6. cat - Display file contents
```

Fig. 29 — The original screenshot shows the display, less, exit and invalid-choice branches.

```
Session  Actions  Edit  View  Help
4. mv      - Move or rename
5. rm      - Remove file
6. cat     - Display file contents
7. less    - View longer files
0. Exit    - 00_files_pipes_redirect.sh

What would you like to do? 3
Enter source file: test.txt
Enter destination: test_copy.txt
SUCCESS: File copied.

=====
FILE & DIRECTORY MANAGER
=====
1. mkdir   - Create directory
2. touch   - Create empty file
3. cp      - Copy file
4. mv      - Move or rename
5. rm      - Remove file
6. cat     - Display file contents
7. less    - View longer files
0. Exit

What would you like to do? 4
Enter source: test_copy.txt
Enter new name/destination: renamed_test.txt
SUCCESS: Move/rename completed.

=====
FILE & DIRECTORY MANAGER
=====
1. mkdir   - Create directory
2. touch   - Create empty file
3. cp      - Copy file
4. mv      - Move or rename
5. rm      - Remove file
6. cat     - Display file contents
7. less    - View longer files
0. Exit

=====
What would you like to do? 6
Enter file name: renamed_test.txt
===== FILE CONTENT =====
=====
=====
=====
FILE & DIRECTORY MANAGER
=====
1. mkdir   - Create directory
2. touch   - Create empty file
3. cp      - Copy file
```

Fig. 30 — The original terminal evidence shows syntax checking and menu interaction.

```
Session  Actions  Edit  View  Help
4. mv      - Move or rename
5. rm      - Remove file
6. cat     - Display file contents
7. less    - View longer files
0. Exit    - 00_files_pipes_redirect.sh

What would you like to do? 4
Enter source: test_copy.txt
Enter new name/destination: renamed_test.txt
SUCCESS: Move/rename completed.

FILE & DIRECTORY MANAGER
1. mkdir   - Create directory
2. touch   - Create empty file
3. cp      - Copy file
4. mv      - Move or rename
5. rm      - Remove file
6. cat     - Display file contents
7. less    - View longer files
0. Exit

What would you like to do? 6
Enter file name: renamed_test.txt
FILE CONTENT
bash_pipeline_lab/topics.txt

FILE & DIRECTORY MANAGER
1. mkdir   - Create directory
2. touch   - Create empty file
3. cp      - Copy file
4. mv      - Move or rename
5. rm      - Remove file
6. cat     - Display file contents
7. less    - View longer files
0. Exit    - bash_pipeline_lab/topics.txt

What would you like to do? █
bash
Linux
bash
Security
Linux
Networking
```

Fig. 31 — The original terminal evidence shows the file and directory operations.

```
Session  Actions  Edit  View  Help

4. mv      - Move or rename
5. rm      - Remove file
6. cat     - Display file contents
7. less    - View longer files
0. Exit    bash_files_pipes_redirect.sh

=====

What would you like to do? 4
Enter source: test_copy.txt
Enter new name/destination: renamed_test.txt
SUCCESS: Move/rename completed.

=====

FILE & DIRECTORY MANAGER

=====

1. mkdir   - Create directory
2. touch   - Create empty file
3. cp      - Copy file
4. mv      - Move or rename
5. rm      - Remove file
6. cat     - Display file contents
7. less    - View longer files
0. Exit

=====

What would you like to do? 6
Enter file name: renamed_test.txt
=====
FILE CONTENT
=====
bash_pipeline_lab/topics.txt

=====

FILE & DIRECTORY MANAGER

=====

1. mkdir   - Create directory
2. touch   - Create empty file
3. cp      - Copy file
4. mv      - Move or rename
5. rm      - Remove file
6. cat     - Display file contents
7. less    - View longer files
0. Exit    bash_pipeline_lab/topics.txt

=====

What would you like to do? 0
Goodbye!

(xterra@kali)-[~/icdfa_bash_lab]
$
```

14. Lab 14 — Debugging & Safe Practice

Objective: Use `bash -n` and `bash -x` to check and trace scripts.

Answers: `bash -n` checks syntax without execution. `bash -x` traces expanded commands and branches. A harmless error can be detected with `bash -n`, corrected, and then tested again.

Fig. 32 — The original screenshot shows the syntax-checking and tracing commands.

```
Session  Actions  Edit  View  Help
GNU nano 9.0                                11_calculator.sh
#!/usr/bin/env bash
# echo > /dev/null & bash_pipeline_redirect.sh

read -r -p "First integer: " a
read -r -p "Second integer: " b

if ! [[ "$a" =~ ^-?[0-9]+$ && "$b" =~ ^-?[0-9]+$ ]]; then
    echo "Please enter integers only."
    exit 1
fi

===== FILE CONTENTS =====
echo "1) Add"
echo "2) Subtract"
echo "3) Multiply"
echo "4) Divide"

===== NUMBER OF LINES =====
read -r -p "Choose operation: " op

case "$op" in
    1)
        echo "Result: $((a + b))"
        ;;
    2)
        echo "Result: $((a - b))"
        ;;
    3)
        echo "Result: $((a * b))"
        ;;
    4)
        if (( b == 0 )); then
            echo "Cannot divide by zero."
            exit 1
        fi
        echo "Result: $((a / b))"
        ;;
    *)
        echo "Invalid operation."
        ;;
esac
exit 0
# echo > /dev/null & bash_pipeline_redirect.sh > bash_pipeline_lab/topics.txt
# echo > /dev/null & bash_pipeline_redirect.sh
```

Fig. 33 — The original terminal evidence shows the traced commands and expanded values.

```
Session  Actions  Edit  View  Help

(xterra@kali)-[~]
$ bash -n 11_calculator.sh

(xterra@kali)-[~]
$ bash -x 11_calculator.sh
+ read -r -p 'First integer: ' a
First integer: 10
+ read -r -p 'Second integer: ' b
Second integer: 5
+ [[ 10 =~ ^-[0-9]+$ ]]
+ [[ 5 =~ ^-[0-9]+$ ]]
+ echo '1) Add'
1) Add
+ echo '2) Subtract'
2) Subtract
+ echo '3) Multiply'
3) Multiply
+ echo '4) Divide'
4) Divide
+ read -r -p 'Choose operation: ' op
Choose operation: 1
+ case "$op" in
+ echo 'Result: 15'
Result: 15

(xterra@kali)-[~]
$ cp 11_calculator.sh 11_calculator_backup.sh

(xterra@kali)-[~]
$ nano 11_calculator.sh

(xterra@kali)-[~]
$ bash -n 11_calculator.sh
11_calculator.sh: line 18: syntax error near unexpected token `newline'
11_calculator.sh: line 18: `case "$op" in ('

(xterra@kali)-[~]
$
```

Fig. 34 — The original screenshot shows the debugging result after the test change.

```
Session  Actions  Edit  View  Help
GNU nano 9.0                                11_calculator.sh *
#!/usr/bin/env bash
# echo $a $b > bash_pipeline_redirect.sh

read -r -p "First integer: " a
read -r -p "Second integer: " b

if ! [[ "$a" =~ ^-?[0-9]+$ && "$b" =~ ^-?[0-9]+$ ]]; then
    echo "Please enter integers only."
    exit 1
fi

===== FILE CONTENTS =====
echo "1) Add"
echo "2) Subtract"
echo "3) Multiply"
echo "4) Divide"

===== NUMBER OF LINES =====
read -r -p "Choose operation: " op

case "$op" in
1)
    echo "Result: $((a + b))"
    ;;
2)
    echo "Result: $((a - b))"
    ;;
3)
    echo "Result: $((a * b))"
    ;;
4)
    if (( b == 0 )); then
        echo "Cannot divide by zero."
        exit 1
    fi
    echo "Result: $((a / b))"
    ;;
*)
    echo "Invalid operation."
    ;;
esac

=====
$ ./11_calculator.sh > bash_pipeline_lab/topics.txt
=====
```

Fig. 35 — The original evidence confirms the corrected script can be checked and run.

```
Session Actions Edit View Help

(xterra@kali)-[~]
$ bash -n 11_calculator.sh

(xterra@kali)-[~]
$ bash -x 11_calculator.sh
+ read -r -p 'First integer: ' a
First integer: 10
+ read -r -p 'Second integer: ' b
Second integer: 5
+ [[ 10 =~ ^-?[0-9]+$ ]]
+ [[ 5 =~ ^-?[0-9]+$ ]]
+ echo '1) Add'
1) Add
+ echo '2) Subtract'
2) Subtract
+ echo '3) Multiply'
3) Multiply
+ echo '4) Divide'
4) Divide
+ read -r -p 'Choose operation: ' op
Choose operation: 1
+ case "$op" in
+ echo 'Result: 15'
Result: 15

(xterra@kali)-[~]
$ cp 11_calculator.sh 11_calculator_backup.sh

(xterra@kali)-[~]
$ nano 11_calculator.sh

(xterra@kali)-[~]
$ bash -n 11_calculator.sh
11_calculator.sh: line 18: syntax error near unexpected token `newline'
11_calculator.sh: line 18: `case "$op" in ('

(xterra@kali)-[~]
$ nano 11_calculator.sh

(xterra@kali)-[~]
$ bash -n 11_calculator.sh

(xterra@kali)-[~]
$ bash 11_calculator.sh
First integer: 23
Second integer: 34
1) Add
2) Subtract
3) Multiply
4) Divide
Choose operation: █
```

15. Lab 15 — Final Project: Bash Student Assistant

Objective: Combine the lab concepts into one useful program.

Analysis: The assistant uses input, functions, if, case, loops, find, pipes, command substitution and arithmetic. It provides system information, file operations and a calculator, repeating until option 0 and preventing division by zero.

Key concepts: functions provide reusable operations; command substitution collects system output; if validates input and files; case selects menu actions; the loop repeats the menu; pipes and arithmetic support file counts and calculations.

Fig. 36 — The original editor screenshot shows the final-project functions, menu, validation and calculator sections.

```

Session  Actions  Edit  View  Help
(xterra@kali)-[~]
$ cd ~/icdfa_bash_lab
(xterra@kali)-[~/icdfa_bash_lab]
$ nano 13_student_assistant_final.sh
(xterra@kali)-[~/icdfa_bash_lab]
$ bash -n 13_student_assistant_final.sh
(xterra@kali)-[~/icdfa_bash_lab]
$ bash 13_student_assistant_final.sh
=====
          ICDFA BASH STUDENT ASSISTANT
=====
Enter your name: Clement

=====
          ICDFA BASH STUDENT ASSISTANT
=====
Hello, Clement
1. Show system information
2. Count files
3. Create a folder
4. Create a file
5. Display a file
6. Calculator
0. Exit
What would you like to do? 2
Files in this directory: 4

=====
          ICDFA BASH STUDENT ASSISTANT
=====
Hello, Clement
1. Show system information
2. Count files
3. Create a folder
4. Create a file
5. Display a file
6. Calculator
0. Exit
What would you like to do? 3
Enter folder name: final_test
SUCCESS: Folder 'final_test' created.

=====
          ICDFA BASH STUDENT ASSISTANT
=====
Hello, Clement
1. Show system information
2. Count files
3. Create a folder
4. Create a file
5. Display a file
6. Calculator
0. Exit
What would you like to do? 4
Enter file name: test.txt
SUCCESS: File 'test.txt' created.

=====
          ICDFA BASH STUDENT ASSISTANT
=====

```

Fig. 37 — The original terminal evidence shows syntax checking and initial execution of the assistant.

```

Session  Actions  Edit  View  Help

What would you like to do? 4
Enter file name: test.txt
SUCCESS: File 'test.txt' created.

=====
          ICDFA BASH STUDENT ASSISTANT
=====

Hello, Clement
1. Show system information
2. Count files
3. Create a folder
4. Create a file
5. Display a file
6. Calculator
0. Exit
What would you like to do? 5
Enter file name: test.txt

=====
          ICDFA BASH STUDENT ASSISTANT
=====

Hello, Clement
1. Show system information
2. Count files
3. Create a folder
4. Create a file
5. Display a file
6. Calculator
0. Exit
What would you like to do? 6
Enter first integer: 20
Enter second integer: 5
1. Add
2. Subtract
3. Multiply
4. Divide
Choose operation: 4
Result: 4

=====
          ICDFA BASH STUDENT ASSISTANT
=====

Hello, Clement
1. Show system information
2. Count files
3. Create a folder
4. Create a file
5. Display a file
6. Calculator
0. Exit
What would you like to do?
11_calculator.sh
First integer: 23
Second integer: 34
1. Add
2. Subtract
3. Multiply
4. Divide
Choose operation:
error near unexpected token 'newline'
"$@" in 1'

```

Fig. 38 — The original screenshot shows menu interaction and the selected assistant operation.

```

Session  Actions  Edit  View  Help
GNU nano 9.0      13_student_assistant_final.sh
echo "          ICDFA BASH STUDENT ASSISTANT"
echo "===== "

read -r -p "Enter your name: " name
if [[ -z "$name" ]]; then
    echo "Name cannot be empty."
    exit 1
fi

while true; do
    show_header [0-9]-$
    echo "Hello, $name"
    echo "1. Show system information"
    echo "2. Count files"
    echo "3. Create a folder"
    echo "4. Create a file"
    echo "5. Display a file"
    echo "6. Calculator"
    echo "7. Show IP Address"
    echo "0. Exit"

    read -r -p "What would you like to do? " choice
    case "$choice" in
        1) system_info
        2) count_files
        3) create_folder
        4) create_file
        5) display_file
        6) calculator
        7) ip -br addr
        0) echo "Goodbye, $name!"
            break
        *) echo "Invalid choice."
    esac
done

```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File [\] Replace ^U Paste ^J Justify [/] Go To Line

Fig. 39 — The original evidence shows further final-project testing and exit behavior.

```
(xterra@kali)-[~/icdfa_bash_lab]
$ nano 13_student_assistant_final.sh

(xterra@kali)-[~/icdfa_bash_lab]
$ bash -n 13_student_assistant_final.sh

(xterra@kali)-[~/icdfa_bash_lab]
$ bash 13_student_assistant_final.sh

=====
ICDFA BASH STUDENT ASSISTANT
=====
Enter your name: Clement

=====
ICDFA BASH STUDENT ASSISTANT
=====
Hello, clement
1. Show system information
2. Count files
3. Create a folder
4. Create a file
5. Display a file
6. Calculator
7. Show IP Address
0. Exit
What would you like to do? 7
lo UNKNOWN 127.0.0.1/8 :: 1/128
eth0 UP 10.0.2.15/24 fe80::a00:27ff:fe3c:3f0f/64

=====
ICDFA BASH STUDENT ASSISTANT
=====
Hello, clement
1. Show system information
2. Count files
3. Create a folder
4. Create a file
5. Display a file
6. Calculator
7. Show IP Address
0. Exit
What would you like to do? █
```

16. Final Assessment Challenge

A suitable `final_assessment.sh` must validate a name and number, repeat a four-choice case menu, use a decision, logical operator, function, command substitution, arithmetic and a pipe, then be checked with `bash -n` and traced with `bash -x`.

Example: Use menu options for system information, squaring a number, file counting with `find . -type f | wc -l`, and exit. Use a function for the menu and `if/elif/else` to classify the number.

17. Student Submission Checklist

- ✓ Labs are arranged in the same order as the practical manual.
- ✓ Original screenshots are retained and placed under their corresponding lab.
- ✓ Variables and user input are explained.
- ✓ `if/elif/else`, comparisons and logical operators are explained.
- ✓ case, loops, functions, pipes, redirection and command substitution are explained.
- ✓ `bash -n` and `bash -x` are explained.
- ✓ The final Bash Student Assistant is analysed.

18. Conclusion

The practical work demonstrates a progressive Bash scripting workflow, beginning with terminal navigation and moving through variables, input, arguments, decisions, logical operators, menus, loops, functions, text processing, file management, debugging and a final integrated assistant. The original terminal and editor evidence has been preserved, while the accompanying explanations answer the stated student tasks and identify the important commands for assessment.

Clement Eneji — C11/26/OCO/17773 — CORE101 Bash Scripting